Banco de Dados de Autenticação no MySQL Workbench

Turma: MTEC - N - Informática Para Internet - 1A

Disciplina: Banco de Dados

Professor: José Rubens

Objetivo e pré-requisitos

Ao final da aula, você terá um banco `auth_db` criado 100% via DDL no MySQL Workbench, com tabelas para usuários, perfis e histórico de login, pronto para ser consumido por uma tela de login.

Pré-requisitos:

- MySQL Server 8.0+ e MySQL Workbench instalados
- Conexão local configurada (ex.: `root@localhost:3306`)
- Noções de tipos de dados, chave primária e chave estrangeira

Competências trabalhadas: DDL (`CREATE`, `ALTER`, `DROP`), constraints (`PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK`, `DEFAULT`), modelagem relacional e boas práticas de armazenamento de senha.

Enunciado

> **Atividade: Banco de dados de autenticação**
>
> Você foi contratado para criar a camada de dados da tela de login de um sistema web. A tela terá os campos **e-mail** e **senha**, um botão **Entrar** e um link **Esqueci minha senha**.
>
> Usando **somente comandos DDL** no MySQL Workbench (sem usar o assistente gráfico de criação de tabelas), crie um banco chamado `auth_db` com:
>
> 1. Uma tabela de **perfis** (ex.: administrador, usuário comum).
> 2. Uma tabela de **usuários** com nome, e-mail único, hash da senha, status ativo/inativo, contador de tentativas falhas e datas de criação/atualização.
> 3. Uma tabela de **histórico de login** que registre cada tentativa (sucesso ou falha), com data, IP e navegador.
> 4. Uma tabela de **tokens de recuperação de senha** com validade.
>
> Siga o passo a passo, execute cada bloco com **Ctrl + Shift + Enter**, confira o resultado no painel **Output** e atualize o **Navigator** (ícone de refresh) para ver os objetos criados. Ao final, salve o script como `auth_db_<seu_nome>.sql` e entregue.
>
> **Regra importante:** a senha **nunca** é armazenada em texto puro. O banco guarda apenas o *hash* gerado pela aplicação (ex.: bcrypt).

Passo a passo

Passo 1 — Abrir o Workbench e um novo script

1. Abra o MySQL Workbench e clique na conexão local.
2. Menu **File > New Query Tab** (ou Ctrl + T).
3. Salve logo de início: **File > Save Script As** → `auth_db_<seu_nome>.sql`.

Passo 2 — Criar o banco de dados

```
-- Remove o banco se já existir (útil para reexecutar o script)
DROP DATABASE IF EXISTS auth_db;

-- Cria o banco com suporte completo a acentos e emojis
CREATE DATABASE auth_db
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

-- Define o banco ativo para os próximos comandos
USE auth_db;
```

Pergunta para discussão: por que usar `utf8mb4` e não `utf8`?

Passo 3 — Tabela de perfis

```
CREATE TABLE perfis (
  id_perfil   TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome        VARCHAR(30)      NOT NULL,
  descricao   VARCHAR(150)     NULL,
  CONSTRAINT pk_perfis PRIMARY KEY (id_perfil),
  CONSTRAINT uq_perfis_nome UNIQUE (nome)
) ENGINE = InnoDB;
```

Passo 4 — Tabela de usuários

```
CREATE TABLE usuarios (
  id_usuario         INT UNSIGNED     NOT NULL AUTO_INCREMENT,
  id_perfil          TINYINT UNSIGNED NOT NULL,
  nome               VARCHAR(100)     NOT NULL,
  email              VARCHAR(150)     NOT NULL,
  senha_hash         VARCHAR(255)     NOT NULL,  -- hash bcrypt/argon2, nunca
a senha
  ativo              BOOLEAN          NOT NULL DEFAULT TRUE,
  tentativas_falhas  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  bloqueado_ate      DATETIME         NULL,
  ultimo_login       DATETIME         NULL,
  criado_em          DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em      DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP
                                      ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_usuarios PRIMARY KEY (id_usuario),
```

```
  CONSTRAINT uq_usuarios_email UNIQUE (email),
  CONSTRAINT fk_usuarios_perfis FOREIGN KEY (id_perfil)
    REFERENCES perfis (id_perfil)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT ck_usuarios_tentativas CHECK (tentativas_falhas <= 10)
) ENGINE = InnoDB;
```

Ponto de atenção: o `UNIQUE` em `email` impede cadastro duplicado e cria um índice que acelera a busca feita no login.

Passo 5 — Tabela de histórico de login

```
CREATE TABLE historico_login (
  id_historico   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario     INT UNSIGNED    NULL,          -- NULL se o e-mail não
existir
  email_digitado VARCHAR(150)    NOT NULL,
  sucesso        BOOLEAN         NOT NULL,
  ip_origem      VARCHAR(45)     NOT NULL,      -- 45 comporta IPv6
  user_agent     VARCHAR(255)    NULL,
  data_hora      DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_historico_login PRIMARY KEY (id_historico),
  CONSTRAINT fk_historico_usuarios FOREIGN KEY (id_usuario)
    REFERENCES usuarios (id_usuario)
    ON DELETE SET NULL
) ENGINE = InnoDB;

CREATE INDEX idx_historico_usuario_data
  ON historico_login (id_usuario, data_hora);
```

Passo 6 — Tabela de tokens de recuperação de senha

```
CREATE TABLE tokens_recuperacao (
  id_token    INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario  INT UNSIGNED NOT NULL,
  token_hash  CHAR(64)     NOT NULL,  -- SHA-256 do token enviado por e-mail
  expira_em   DATETIME     NOT NULL,
  usado       BOOLEAN      NOT NULL DEFAULT FALSE,
  criado_em   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_tokens PRIMARY KEY (id_token),
  CONSTRAINT uq_tokens_hash UNIQUE (token_hash),
  CONSTRAINT fk_tokens_usuarios FOREIGN KEY (id_usuario)
    REFERENCES usuarios (id_usuario)
    ON DELETE CASCADE
) ENGINE = InnoDB;
```

Passo 7 — Ajuste com ALTER TABLE

O cliente pediu um campo de telefone opcional. Adicione sem recriar a tabela:

```
ALTER TABLE usuarios
  ADD COLUMN telefone VARCHAR(20) NULL AFTER email;
```

Passo 8 — Gerar o diagrama (EER)

1. Menu **Database > Reverse Engineer**.
2. Selecione a conexão e o schema `auth_db`.
3. Confirme as etapas até o diagrama aparecer.
4. Tire um print do diagrama e anexe à entrega.

Relacionamentos esperados: `perfis` 1:N `usuarios`; `usuarios` 1:N `historico_login`; `usuarios` 1:N `tokens_recuperacao`.

Verificação

Execute os comandos abaixo para confirmar que a estrutura está correta. Eles usam DML apenas para testar o DDL.

```
-- 1. Conferir as tabelas criadas
SHOW TABLES;

-- 2. Conferir a estrutura de uma tabela
DESCRIBE usuarios;
SHOW CREATE TABLE usuarios;

-- 3. Dados de teste
INSERT INTO perfis (nome, descricao) VALUES
  ('admin', 'Acesso total'),
  ('usuario', 'Acesso comum');

-- Hash bcrypt de exemplo (senha "123456" — apenas para aula)
INSERT INTO usuarios (id_perfil, nome, email, senha_hash) VALUES
  (1, 'Ana Souza', 'ana@escola.com',
   '$2y$10$e0MYzXyjpJS7Pd0RVvHwHeFX4Fh3v0Ztk1v5mU5u3qYp1K1QkG6aC');

-- 4. Consulta que a tela de login fará
SELECT id_usuario, nome, senha_hash, ativo, bloqueado_ate
FROM usuarios
WHERE email = 'ana@escola.com';
```

Testes de constraint (devem dar erro):

- [ ] Inserir outro usuário com `ana@escola.com` → erro 1062 (UNIQUE)
- [ ] Inserir usuário com `id_perfil = 99` → erro 1452 (FOREIGN KEY)
- [ ] Atualizar `tentativas_falhas` para 15 → erro 3819 (CHECK)
- [ ] Excluir o perfil `admin` com usuário vinculado → erro 1451 (RESTRICT)